EBERHARD KARLS UNIVERSITÄT TÜBINGEN

Department of Computer Science - Database Systems

SQL as a Programming Language

Implementing Distance-Vector Routing with Recursive CTEs in DuckDB

Mehdi Hosseini

Master Seminar: Database Systems - Summer Semester 2026

Motivation

Can a declarative query language simulate a stateful, distributed network protocol?

The Challenge

- Standard recursive CTEs are **append-only**
- DVR needs **in-place state updates**

The Solution

DuckDB's `USING KEY` enables **stateful recursion** inside SQL

Distance-Vector Routing Protocol

Core Idea

- Each node maintains a **cost vector**
- Neighbors **exchange** tables periodically
- Costs refined when cheaper path found

Key Properties

- **Distributed:** No central coordinator
- **Iterative:** Round-by-round updates
- **Convergent:** Shortest paths in $\leq |V| - 1$ steps

Algorithm Steps

- **T=0:** Direct neighbor discovery
- **T=1:** First exchange and relaxation
- **T>=2:** Repeat until fixpoint

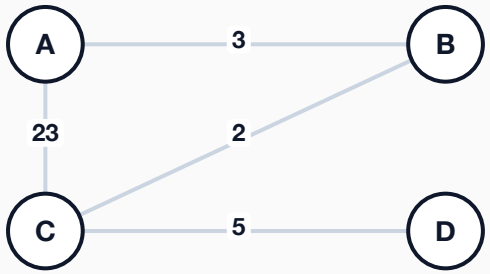
Real-World Protocols

- **RIP:** Classic DVR (max 15 hops)
- **EIGRP:** Cisco hybrid protocol
- **BGP:** Internet path-vector routing

The Bellman-Ford Equation

$$d(u, v) = \min_{w \in \text{Adj}(u)} \{\text{cost}(u, w) + d(w, v)\}$$

Example Network Topology



Direct link A→C costs 23, but indirect path A→B→C costs only 3+2 = **5**.

Step-by-Step Convergence

Round	Discovery from Router A
T=0	B=3, C=23 (direct neighbors)
T=1	C=5 via B (3+2 < 23), D=28 via C
T=2	D=10 via B (3+2+5 < 28)
T=3	No changes - **converged**

SQL Recursive CTEs: The Challenge

Standard SQL: The Problem

- **Append-only:** Rows accumulate via `UNION ALL`
- **No updates:** Cannot modify rows mid-recursion
- **Path explosion:** All walks enumerated
- **Memory limit:** SQLite fails beyond 11 nodes

DVR Requirement

- **In-place updates:** Overwrite with better paths
- **Bounded state:** One route per pair, $O(V^2)$
- **Fixpoint:** Stop when no more updates

Mismatch: CTEs grow history; DVR needs fixed-size iterative refinement.

DuckDB `USING KEY`: The Solution

`USING KEY (from_node, to_node)`: Matching keys **replace** old rows instead of appending.

In-Place Updates

One entry per router pair. Cheaper paths overwrite.

Bounded Memory

Working set stays at $O(V^2)$. No walk explosion.

Auto Fixpoint

Halts when delta set is empty.

Memory Usage	Walk Explosion	Termination	Post-Hoc Agg.
O(V²)	None	Automatic	Not needed

Standard CTE vs Keyed CTE

SQLite: Standard (Append-Only)

sqlite_standard.sql — SQL

```sql
WITH RECURSIVE bellman(s,t,cost,hop)
AS (
  SELECT from_node, to_node,
         cost, to_node
  FROM edges
  UNION ALL
  SELECT b.s, e.to_node,
         b.cost + e.cost, b.hop
  FROM bellman b
  JOIN edges e ON b.t = e.from_node
  WHERE b.s <> e.to_node
)
-- POST-HOC aggregation required
SELECT s, t, MIN(cost), hop
FROM bellman GROUP BY s, t
```

DuckDB: Keyed (USING KEY)

duckdb_keyed.sql — SQL

```sql
WITH RECURSIVE routing(s,t,cost,hop)
    USING KEY (s, t) AS (
  SELECT from_node, to_node,
         cost, to_node
  FROM edges
  UNION
  SELECT r.s, e.to_node,
         MIN(r.cost + e.cost),
         arg_min(r.hop,
                 r.cost + e.cost)
  FROM routing r
  JOIN edges e ON r.t = e.from_node
  WHERE r.s <> e.to_node
  GROUP BY r.s, e.to_node
)
SELECT * FROM routing
```

Key Difference: Standard CTEs accumulate all walks (post-hoc `GROUP BY`). `USING KEY` prunes **during** recursion.

The Algorithmic Core

duckdb_dvr_simulation.sql — SQL

```sql
WITH RECURSIVE routing_table(
  from_node, to_node,
  best_cost, next_hop)
  USING KEY (from_node, to_node) AS (
  -- T=0: Direct neighbor discovery
  SELECT from_node, to_node,
         cost, to_node AS next_hop
  FROM edges
  UNION
  -- Bellman-Ford relaxation
  SELECT r.from_node, e.to_node,
    MIN(r.best_cost + e.cost),
    arg_min(r.next_hop,
            r.best_cost + e.cost)
  FROM routing_table AS r
  JOIN edges AS e
    ON r.to_node = e.from_node
  LEFT JOIN recurring.routing_table
    AS ex ON ex.from_node = r.from_node
       AND ex.to_node = e.to_node
  WHERE r.from_node <> e.to_node
    AND (ex.best_cost IS NULL
      OR r.best_cost + e.cost
         < ex.best_cost)
  GROUP BY r.from_node, e.to_node
)
SELECT * FROM routing_table
ORDER BY from_node, to_node;
```

Line-by-Line Review

- **`USING KEY`:** Stateful row updates.
- **Base case:** Direct neighbor costs.
- **`JOIN edges`:** Neighbor propagation.
- **`recurring.*`:** Bounded state access.
- **`IS NULL OR <`:** Relaxation condition.
- **`MIN` + `arg_min`:** Select cheapest hop.

One query replaces hundreds of lines of imperative code.

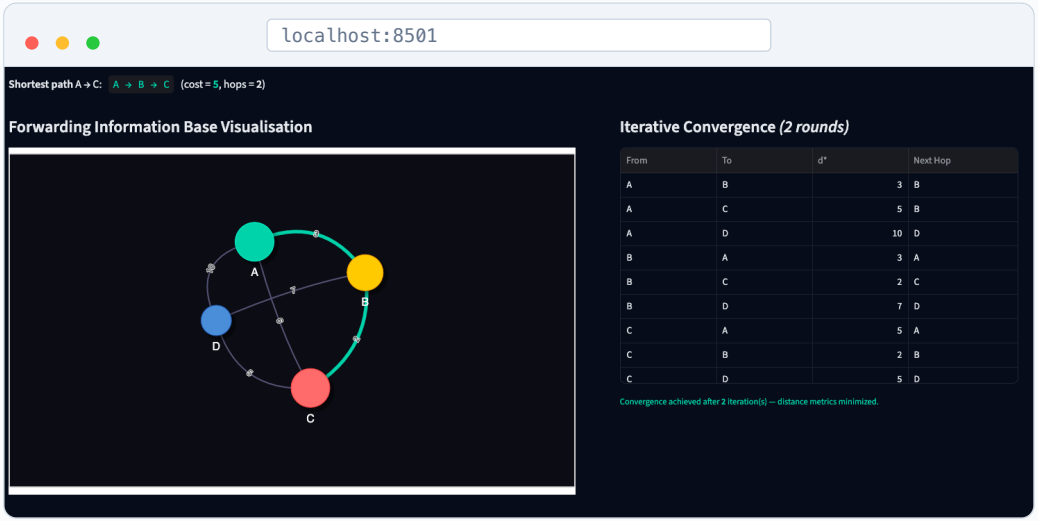
Convergence Results: Full Network

Converged Forwarding Table

From	To	Cost	Next Hop
A	B	**3**	B
A	C	**5**	B
A	D	**10**	B
B	A	**3**	A
B	C	**2**	C
B	D	**7**	C
C	A	**5**	B
C	B	**2**	B
C	D	**5**	D
D	A	**10**	C
D	B	**7**	C
D	C	**5**	C

Observations

- **A→C = 5 (via B):** Indirect beats direct (23).
- **A→D = 10 (via B):** Multi-hop path.
- **Convergence:** 3 rounds for 4 nodes.



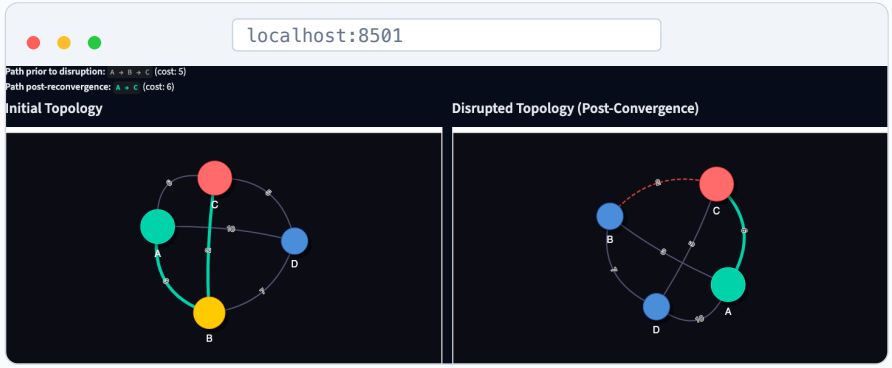
Network Failure and Re-convergence

Simulating Link Failure

simulate_failure.sql — SQL

```sql
DELETE FROM edges
WHERE (from_node='B' AND to_node='C')
   OR (from_node='C' AND to_node='B');
```

Re-run query to discover next-best paths.

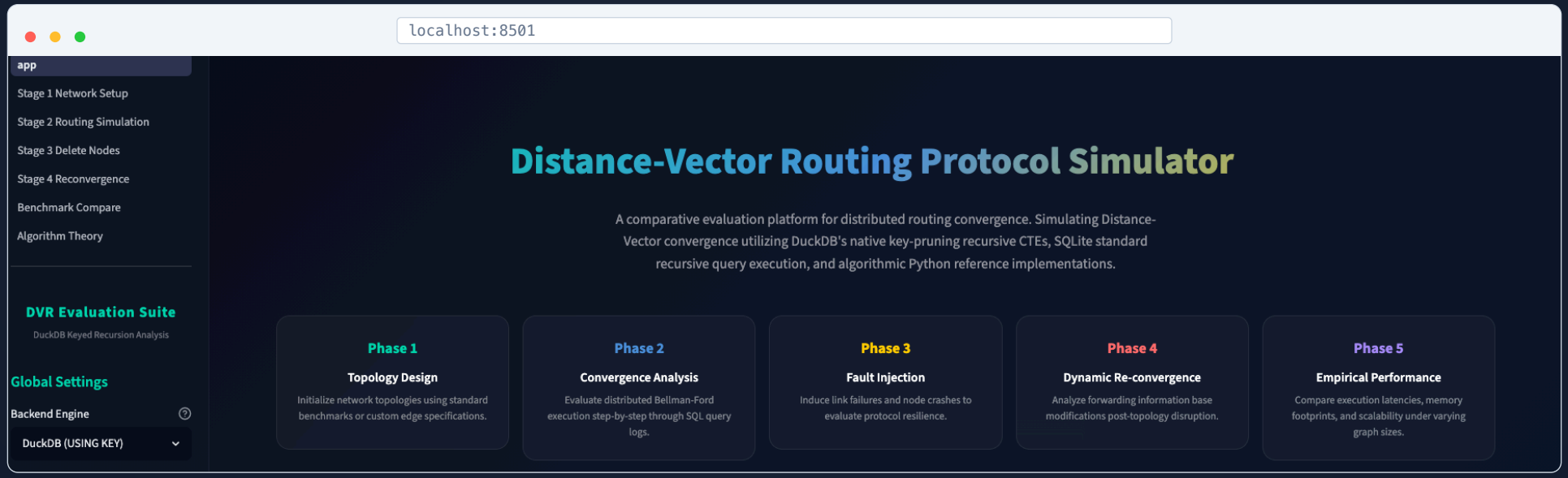


Routing Table Shift

Route	Before	After	Δ
A ⟶ C	5	**6**	+1
B ⟶ C	2	**9**	+7
C ⟶ B	2	**9**	+7

INTERACTIVE DEMONSTRATION

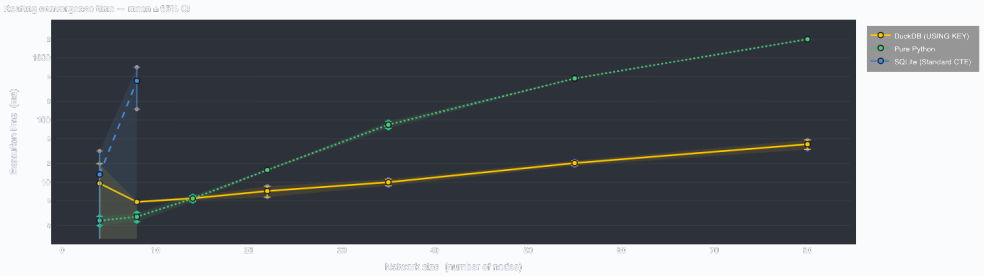
Live Demo

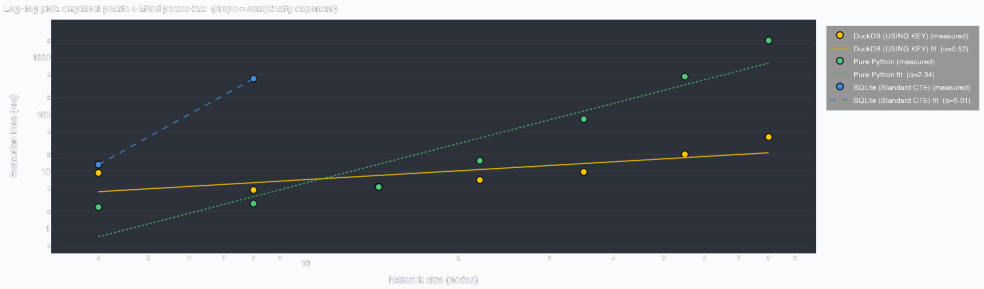


Interactive Streamlit Simulator

Streamlit · DuckDB · Pyvis · Plotly

Benchmark Results





Key Findings

- **DuckDB:** 5-15x faster than SQLite
- **SQLite:** Fails beyond 11 nodes
- **DuckDB:** Polynomial $O(V \cdot E)$
- **Python:** Baseline comparison

Backend	Complexity	Max N
DuckDB USING KEY	O(V·E)	50+
SQLite Standard	O(V·E·P)	11
Python Bellman-Ford	O(V²·E)	50+
Python Dijkstra	O(V(V+E)logV)	50+
Python Floyd-Warshall	O(V³)	50+

Key Findings

1. SQL as Algorithm

One recursive query replaces hundreds of lines of imperative code.

2. State-Aware Recursion

No path explosion. $O(V^2)$ working set. 5-15x faster than SQLite.

3. Auto Re-Convergence

`DELETE` a link, re-run query. No code changes needed.

Central Result: With `USING KEY`, SQL becomes a first-class language for simulating distributed protocols.

Conclusion and Future Work

Conclusion

- **1:1 mapping** between SQL recursion and router behavior
- Interactive evaluation suite with 6 phases
- Keyed CTEs: **polynomial**; standard CTEs: **exponential**
- More concise and verifiably correct

Future Research

- **BGP:** Path-vector via SQL arrays
- **OSPF:** Dijkstra in recursive SQL
- **Dynamic Traffic:** Real-time edge metrics
- **Large-Scale:** Internet AS-graph tests

References:
[1] R. Bellman, “On a Routing Problem,” *Q. Appl. Math.*, 1958.
[2] DuckDB Documentation, “Recursive CTEs with USING KEY,” 2026.
[3] A. S. Tanenbaum, *Computer Networks*, 6th ed., Pearson, 2021.

END OF PRESENTATION

Thank You

Questions and Discussion

Mehdi Hosseini

Database Systems Seminar - University of Tübingen - Summer 2026